JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent.atomic

Class DoubleAdder

java.lang.Object
    java.lang.Number
        java.util.concurrent.atomic.DoubleAdder

All Implemented Interfaces:

Serializable

```
public class DoubleAdder
extends Number
implements Serializable
```

One or more variables that together maintain an initially zero `double` sum. When updates (method `add(double)`) are contended across threads, the set of variables may grow dynamically to reduce contention. Method `sum()` (or, equivalently `doubleValue()`) returns the current total combined across the variables maintaining the sum. The order of accumulation within or across threads is not guaranteed. Thus, this class may not be applicable if numerical stability is required, especially when combining values of substantially different orders of magnitude.

This class is usually preferable to alternatives when multiple threads update a common value that is used for purposes such as summary statistics that are frequently updated but less frequently read.

This class extends `Number`, but does *not* define methods such as `equals`, `hashCode` and `compareTo` because instances are expected to be mutated, and so are not useful as collection keys.

Since:

1.8

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`DoubleAdder()`	Creates a new adder with initial sum of zero.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`void`	`add(double x)`	Adds the given value.
`double`	`doubleValue()`	Equivalent to `sum()`.
`float`	`floatValue()`	Returns the `sum()` as a `float` after a narrowing primitive conversion.
`int`	`intValue()`	Returns the `sum()` as an `int` after a narrowing primitive conversion.
`long`	`longValue()`	Returns the `sum()` as a `long` after a narrowing primitive conversion.
`void`	`reset()`	Resets variables maintaining the sum to zero.
`double`	`sum()`	Returns the current sum.
`double`	`sumThenReset()`	Equivalent in effect to `sum()` followed by `reset()`.
`String`	`toString()`	Returns the String representation of the `sum()`.

Methods inherited from class java.lang.Number

`byteValue`, `shortValue`

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

DoubleAdder

```
public DoubleAdder()
```

Creates a new adder with initial sum of zero.

Method Detail

add

```
public void add(double x)
```

Adds the given value.

Parameters:

`x - the value to add`

sum

```
public double sum()
```

Returns the current sum. The returned value is *NOT* an atomic snapshot; invocation in the absence of concurrent updates returns an accurate result, but concurrent updates that occur while the sum is being calculated might not be incorporated. Also, because floating-point arithmetic is not strictly associative, the returned result need not be identical to the value that would be obtained in a sequential series of updates to a single variable.

Returns:

`the sum`

reset

```
public void reset()
```

Resets variables maintaining the sum to zero. This method may be a useful alternative to creating a new adder, but is only effective if there are no concurrent updates. Because this method is intrinsically racy, it should only be used when it is known that no threads are concurrently updating.

sumThenReset

```
public double sumThenReset()
```

Equivalent in effect to `sum()` followed by `reset()`. This method may apply for example during quiescent points between multithreaded computations. If there are updates concurrent with this method, the returned value is *not* guaranteed to be the final value occurring before the reset.

Returns:

the sum

toString

```
public String toString()
```

Returns the String representation of the `sum()`.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the `sum()`

doubleValue

```
public double doubleValue()
```

Equivalent to `sum()`.

Specified by:

`doubleValue` in class `Number`

Returns:

the sum

longValue

```
public long longValue()
```

Returns the `sum()` as a `long` after a narrowing primitive conversion.

Specified by:

`longValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type long.

intValue

```
public int intValue()
```

Returns the `sum()` as an `int` after a narrowing primitive conversion.

Specified by:

`intValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type int.

floatValue

```
public float floatValue()
```

Returns the `sum()` as a `float` after a narrowing primitive conversion.

Specified by:

`floatValue` in class `Number`

Returns:

the numeric value represented by this object after conversion to type float.

contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.